LLM Orchestration Across Models and Regions

Project description: Lines 242, 244 and 246

FIELD OF SCIENCE OR TECHNOLOGY	CLAIM PERIOD	PREPARED BY	LENGTH (242 / 244 / 246)
Software engineering (AI systems and distributed architecture)	FY2025	Judah Paemka	326 / 603 / 269 words

About this sample. *This narrative is based on an anonymized, composite project. Client names, internal system names and some figures have been changed or generalized to protect client confidentiality. It demonstrates technical writing and is not a filed claim.*

LINE 242

Scientific or Technological Uncertainties

What scientific or technological uncertainties did you attempt to overcome?

326 of 350 words

We operate a multi-tenant feedback and experience platform whose generative AI features run on Node.js and Kubernetes (Amazon EKS). At the outset, our orchestration framework made stateless, single-step calls through AWS Bedrock. We wanted downstream services to receive a consistent JSON response while allowing tenants to use different foundation models across six geographic pods. Routing had to respect regional availability, sub-processor consent and each client's model opt-in. The difficulty was that responses varied both when an identical prompt was replayed against one model version and when a different model family was substituted.

We drew on provider SDKs, orchestration libraries including LangChain and LlamaIndex, response normalization through LiteLLM, and vendor agent runtimes. These offered mechanisms for calling models, constructing prompts and handling output. They did not establish how our required combination of reasoning tokens, tool-call deltas, guardrail verdicts, structured payloads and errors would behave across the models and configurations we needed. Our knowledge of the existing single-step pipeline did not resolve those interactions. Schema validation and authorization checks are well understood when a complete, deterministic response is available. The approaches we evaluated did not establish how these checks would behave when output arrived incrementally, varied between runs of the same model, and had to carry a user's authorization through tool calls initiated by the model.

We needed to determine whether provider responses could be mapped to one typed contract that remained usable under replay variation and model substitution. Enforcing that contract on a live stream introduced a further problem: waiting for the complete response would remove the latency benefit of streaming, while releasing incomplete structured data could expose consumers to malformed output. Tool calling also required user authorization to survive model-directed calls across regions and pods. We did not know whether the framework could preserve that context at each call boundary while handling regional model unavailability and provider failures. These questions required us to test response structure, streaming behaviour and tool execution together under the platform's constraints.

LINE 244

Work Performed

What work did you perform in the tax year to overcome the scientific or technological uncertainties described in Line 242?

603 of 700 words

At the start of FY2025, we measured response variability using 2,000 production-representative prompts. We replayed identical prompts against the same model version, then substituted model families while holding prompts constant. Schema validation found violations in about 4 percent of same-model replays and 17 percent of cross-family responses. Both caused downstream failures. We first hypothesized that stricter prompt instructions and inference tuning would keep responses within the schema. They reduced the rates only to roughly 3 and 14 percent, so we rejected that hypothesis for the configurations tested.

We next hypothesized that per-model adapters mapping native responses to a canonical contract, with explicit states for thinking, final answers, tool calls, structured payloads and errors, would hold that contract regardless of model. Adapter output still contained malformed payloads, so we added bounded repair: one targeted re-prompt followed by a tolerant parser that extracted the first valid subtree and discarded trailing output. Invalid responses reaching consumers fell to 0.3 percent. Repair added a median 380 milliseconds when triggered. This substantially improved conformance, but did not eliminate downstream contract failures.

We also tested the assumption that models producing the same contract would behave equivalently. Running identical agents across models exposed differences in accuracy, latency and schema adherence. We recorded these as per-model capability declarations. A consumer generating deeply nested filter JSON still produced semantic errors after model substitution, so we pinned its agent to one model and retained the abstraction for controlled future swaps. Profiling attributed about 85 percent of end-to-end latency to model reasoning. Prefetching and decomposition were neutral or slower, which limited their usefulness for reducing response time.

For streaming, we hypothesized that structured payloads could be validated mid-stream without buffering the whole response. We streamed typed items over Server-Sent Events (SSE), and the framework detected candidate structured payloads early, buffering and validating them while free text continued to stream. Time to first token remained below 1.2 seconds in these tests. Schema adherence nevertheless deteriorated over longer conversations. Adding a bounded schema reminder on each turn restored adherence in the tested conversations without exhausting the context window.

We then hypothesized that authorization could be enforced at the framework's tool boundary rather than trusted to the model or a vendor runtime. In adversarial sessions across tenants and regions, models attempted tools outside the tenant's entitlement in about 3 percent of cases. A runtime registry resolved available tools from tenant configuration, and the framework re-asserted user context and trace identifiers at every tool boundary. It rejected every unauthorized call in those tests, including syntactically valid requests. Explicit provider-failure states, retries and backoff distinguished transient Bedrock or network faults from schema drift. Per-tenant, per-region arbitration across six pods used defined fallbacks when a model was unavailable, keeping those availability changes within the response contract.

Partial edits in the block-based content editor exposed a further limitation. A constrained block schema still yielded wrapper noise, unsupported blocks and ordering defects. We hypothesized that referencing protected blocks by stable identifiers, rather than passing their content to the model, would preserve them through edits. We replaced protected non-text blocks with context blocks referenced by globally unique

identifiers (GUIDs), instructed the model to echo them unchanged, and rehydrated the originals on return. Addressing blocks by GUID rather than position preserved their identity through edits. Reinvocations fell from about 9 percent to under 2 percent of requests.

By year end, the tests supported the streaming and authorization approaches for the models examined. Response conformance had improved substantially, although the residual 0.3 percent invalid-response rate and semantic errors after some model swaps limited the result. Automated contract testing and regional model audits continue into FY2026.

LINE 246

Scientific or Technological Advancements

What scientific or technological advancements did you achieve or attempt to achieve as a result of the work described in Line 244?

269 of 350 words

The experiments showed why our initial prompt-based approach was insufficient. For the models and configurations tested, stronger instructions and inference tuning reduced schema errors but left responses unreliable for downstream use. Mapping native output to explicit contract states, then validating and repairing it, reduced invalid responses reaching consumers to 0.3 percent. Downstream services could use a common interface, but that residual failure rate meant we had not achieved complete contract conformance.

A common response shape also did not make models interchangeable. Tests with identical agents exposed differences in accuracy and latency, while deeply nested filter generation remained sensitive to model substitution. This gave us a basis for recording capabilities separately and retaining a pinned model where semantic correctness did not survive a swap.

Streaming tests established a workable boundary between free text and structured output. Detecting and validating candidate payloads while continuing free-text delivery kept time to first token below 1.2 seconds in the tested conditions. Longer conversations required a bounded schema reminder on each turn to maintain adherence.

Adversarial tool tests showed that a syntactically valid request could still exceed a tenant's entitlement. Resolving tools from tenant configuration and checking user context at each boundary rejected all such attempts in the test set. For partial content edits, GUID references allowed the framework to retain protected blocks and reduced the need to invoke the model again.

These findings define the conditions under which the framework can support a common interface and where model-specific restrictions remain necessary. Residual contract failures, untested model families and changes to pinned model versions require further investigation. Contract tests and regional audits continue in FY2026.